Technical Report: CarBench-IJCAI

Jakob Wagner[1,2] , **Erik Voigt**[2] ,
[1]BMW Group
[2]Otto-Friedrich-University Bamberg
jakob.wagner@bmw.de, erik.voigt@uni-bamberg.de

Abstract

We present a knowledge graph based retrieval system for tool selection in LLM agents, evaluated on CAR-bench. Instead of treating capabilities as a flat list, we encode tools, parameters, outputs, policies, and examples as a typed graph and use personalized PageRank to surface relevant tools for a given query. Multiple correction steps adjust structural drawbacks of our capability knowledge graph and stalling behavior of agents. The retrieved tools are enriched with connected policies, and tools before being passed to the model, giving it more context to reason about feasibility and constraints rather than hard coding these decisions into the retrieval layer. Our results show that graph structured, policy aware retrieval improves tool selection over flat baselines in two out of the three settings posed in this challenge.

1 Introduction

Our approach aims to improve the tool/information retrieval of the agentic system. Tool retrieval is the problem of selecting a useful subset of capabilities from a catalog. It is part of the greater scope of tool-learning. We treat this sub-step as a first-class concept within tool learning: the decision mechanism by which a system matches a user query to available capabilities before it makes any concrete calls to those capabilities. We also adapt parts of the remaining pipeline, but they are not central to our approach.

2 Architecture

The foundation of our approach is a retrieval system built over a precomputed Knowledge Graph (KG) that encodes the CAR-bench domain. The agent makes no binary decisions in the retrieval layer. It does not assert "this tool is right" or "this policy applies"; it surfaces the most relevant subset of the KG and passes that subset to the model together with enough context for the model to reason.

In the KG, nodes represent entities at several levels of abstraction (cf. fig. 1): individual tools, the functional domains to which tools belong, the parameters each tool accepts, the structured outputs each tool produces, domain policies extracted from the CAR-bench wiki, logic clauses that decompose complex policies, worked examples drawn from the training split, and the individual action steps within those examples. Edges encode the relationships among these entities: a tool belongs to a domain, accepts parameters, produces outputs, serves as the source or target of a policy rule, and appears in example trajectories. Edges are typed and weighted, and stronger structural relationships carry greater weight during propagation.

We construct the underlying KG in four steps: (i) we extract each tool's signature—name, description, and input/output parameters—together with the directory of its source file, which we refer to as its *domain*; (ii) we use a Large Language Model (LLM) to extract policies from the car-bench [KSA26] wiki, linking each policy to its corresponding tool(s); (iii) we process the training examples, connecting each step to the tool it invokes; and (iv) finally, we generate embeddings for all nodes and assign weights to both edges and nodes. Figure 1 shows the conceptual layout of our KG.

Our approach follows three steps: (i) we retrieve a set of seed nodes using cosine similarity between the query embedding and node embeddings as well as state nodes and the target tools; (ii) we run Personalized PageRank (PPR) over the weighted KG to propagate relevance from these seeds; and (iii) we return the top-k tool matches to the user, each enriched with the KG structure by adding information on connected policies and examples. Figure 2 shows the schematic procedure for our agent.

2.1 Seed retrieval

We embed the user's query and compare it against every node in the graph by cosine similarity. We retrieve seeds separately per node type (tools, outputs, parameters, domains, rules, examples) and combine the per-type results with type-specific weights. Tool nodes receive the highest weight as the primary decision variable; output and parameter nodes receive intermediate weight, naming the concepts a task requires; rule nodes receive moderate weight, since policies are often relevant but rarely the dominant signal; examples receive the lowest weight, since their relevance is task-specific.

Per-slot retrieval matters because a single query touches multiple node types. The query "navigate to the nearest charging station" may match the term "charging" in a policy node, the term "navigate" in a parameter description node, and the sequence `get_current_location` →

`start_navigation` in an example node. Retrieving from each type independently prevents any one high-scoring type from crowding out the others. We then normalize the seed set between 0 and 1.

We complete the seed set with two additional sources: the agent's last states and its target tools. The last state is the set of tools returned in the previous turn; we add each past state as a seed, weighted by exponential decay with respect to the number of turns since it occurred. Target definition is handled using a tool to set the goal. This tool is called in the initial turn and can be called to adjust the target at any point in time by the agent. The previous states stabilize retrieval, while the target node draws the agent state towards the final goal.

2.2 Personalized PageRank

The seeds define the personalized state we use for PPR. PPR distributes random-walk probability mass that is initially concentrated at the seeds and diffuses through the edge structure. Because edges encode structural relationships, relevance propagates to tools connected to relevant parameters, to policies connected to relevant tools, and to examples connected to relevant action sequences, even when those downstream nodes are themselves not similar to the query embedding. We omit the direction of the edges for this case, as we are interested in incoming and outgoing edges. The damping factor controls how quickly relevance decays with graph distance. A high value (default 0.85) keeps the distribution near the structural neighborhood of the seeds, which suits a well-connected domain graph.

Hub nodes (domains, broadly applicable parameters, and general policies) connect to many tools. Under propagation, they accumulate high PPR mass by virtue of their position rather than their relevance to a given query, and a node adjacent to many tools is less informative about which tool to call than a node adjacent to only one. We employ specificity scoring to correct for this bias by discounting nodes with high weighted degree and a high count of adjacent tools, boosting nodes uniquely associated with few tools relative to structural hubs. This step yields a ranked list of tool nodes ordered by their PPR score. We then adjust the scores of tools the agent has already called multiple times. This lets the agent call a tool repeatedly when needed, but also lets it break out of a stalled pattern and try something new.

2.3 What Is Provided to the Language Model

The pipeline emits a prompt block appended to the model's context as a system message. We frame the block as advisory: it instructs the model to treat the hints as relevant only insofar as they are consistent with system policy and the current turn. We deliberately give the model more information than it strictly requires and let it reason about relevance, rather than hard-wiring decisions into the retrieval layer.

We attach to each retrieved tool the policy and dependency rules directly connected to it in the KG. A policy node carries a rule identifier, a type (dependency or constraint), and a human-readable description extracted from the CAR-bench wiki. Attaching policies per tool, rather than as a global list, keeps the model's search space small: it reads only the policies relevant to the candidate tools instead of matching all policies to tools, and it keeps constraints in close token-proximity to the tools they govern.

2.4 Additional Changes

We added a comprehensive system prompt to sway the agent to follow the policies and patterns outlined by the challenge description. We added harnesses that check the time formats against the expected shape and assertions that block calls to tools we did not provide or calls missing required parameters.

3 Results

Table 1 shows the performance of the models. Since our pipeline builds on top of Gemini 2.5 Flash, the Gemini 2.5 Flash row is the relevant baseline for measuring the effect of our full pipeline, which combines the KG+PPR retrieval layer with the system prompt and harness changes described in section 2.4. We did not run an ablation that isolates the retrieval layer from these other changes, so the comparisons below describe the effect of the pipeline as a whole rather than of any single component.

Relative to the Gemini 2.5 Flash baseline, our pipeline raises Base Pass^3 by 22 percentage points ($48\% \rightarrow 70\%$) and Disambiguation Pass^3 by 26 percentage points ($22\% \rightarrow 48\%$). Averaged across all categories, Avg Pass^3 improves by 14 percentage points ($34\% \rightarrow 48\%$) and Avg Pass@3 improves by 31 percentage points ($48\% \rightarrow 83\%$). Hallucination Pass^3 declines by 6 percentage points ($32\% \rightarrow 26\%$).

Our interpretation of the results is that grounding tool selection in retrieved KG context helps identify the correct capability more consistently on tasks where a valid tool exists, while providing no comparable benefit, and possibly a cost, on tasks that require recognizing that no tool applies. We treat this as a hypothesis rather than an established mechanism, since we cannot separate the contribution of the retrieval layer from the system prompt and harness changes with the data collected so far.

Our Disambiguation Pass^3 score (48%) is close to Claude Opus 4.6's (46%). It does suggest that a much smaller model paired with our pipeline can reach parity with a frontier model on disambiguation tasks, which are largely about following explicit instructions and signaling uncertainty rather than about broad reasoning capacity. The gap to Claude Opus 4.6 in Avg Pass^3 (48% versus 58%) is concentrated in Hallucination Pass^3, where our score trails by 22 percentage points, compared to a 10-point gap on Base Pass^3 and a 2-point lead on Disambiguation Pass^3. By category weight, Hallucination accounts for roughly three quarters of the overall deficit and Base for most of the remainder, so the gap is concentrated but not confined to a single category.

The Hallucination Gap Hallucination Pass^3 is 26%, close to the bottom of the field, only slightly below Qwen3-32B (27%). This is the largest single contributor to the overall ranking deficit and stands in contrast to the pipeline's above-median performance on base and disambiguation tasks. One possible explanation is that the retrieved context we attach to each tool, including references to parameters or tools we excluded on purpose from the candidate set, gives the model material it cannot reliably distinguish from genuinely available

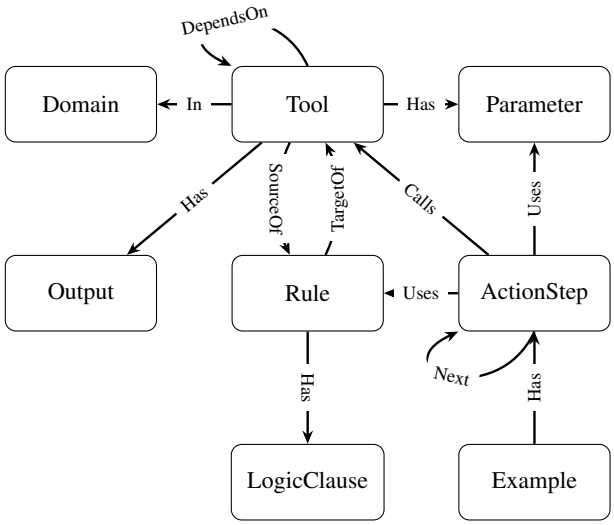

Figure 1: KG meta-structure.

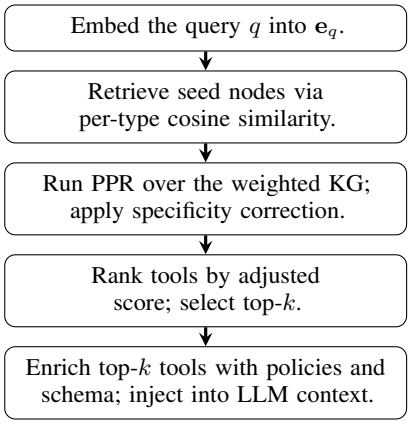

Figure 2: Retrieval pipeline. Query embeddings seed a personalized PageRank walk over the KG; the resulting ranked tool list, together with policies and schemas, is injected into the model's context.

Table 1: Pass^3 consistency (%) on CAR-bench by task type. Base, Hallucination, and Disambiguation columns report per-category scores. Values marked with † are as reported by the original authors and were not reproduced by us. Dashes indicate that the technique can't evaluate this metric or is not reported by Kirmayr, Stappen, and André [KSA26]. The cases indicate the classification according to section 3.

Method	Avg Pass^3	Avg Pass@3	Base Pass^3	Hall Pass^3	Disamb. Pass^3
Claude Opus 4.6† (thinking)	**58**%	79%	**80**%	**48**%	46%
Gemini 2.5 Flash†	34%	52%	48%	32%	22%
Qwen3-32B† (thinking)	31%	66%	45%	27%	22%
Ours (KG + PPR; Gemini 2.5 Flash)	48%	**83**%	70%	26%	**48**%

options. We have not verified this by inspecting failure cases, so it remains a hypothesis to test in future work rather than a confirmed cause.

A second observation, independent of category, is the gap between Pass@3 (83%) and Pass^3 (48%), a difference of 35 percentage points. This gap is larger than the corresponding gap in any baseline model, which ranges from about 9 to 16 percentage points. Pass^3 requires all three trials to succeed, so this variance lowers Pass^3 even when the average single-trial success rate is high. We read this as a reliability weakness of the pipeline on repeated attempts, not only as an artifact of the metric. Reducing this gap, for example through self-consistency sampling, is a direction for future work; we have not measured whether the added inference cost of such techniques would remain below the cost of running a larger model.

4 Related Work

Conventionally, all capabilities are presented to an agentic system as a flat list [Mud+26], from which the LLM selects a capability and supplies the required input parameters. This approach fails to model inter-capability structure, domain hierarchy, or constraints, and it scales poorly: as the number of capabilities grows, retrieval accuracy degrades [NZ+25; Par+25; Mud+26; GS25], the input token budget is strained or exceeded [Par+25; Mud+26], and long-context noise impairs even state-of-the-art models [Luo+25; Liu+25; Mo+25].

Graph-based methods add explicit relations between capabilities, tools, and domains, ranging from training-free retrievers [Lum+25a; LDZ25; Lum+25b; Liu+24a; Liu+24b] to learned Graph Neural Network (GNN) retrievers [NZ+25]. Training-free systems typically construct a capability or tool KG from descriptions, input-output signatures, or usage traces, and then exploit graph structure for multi-step planning [Liu+24b; Lum+25a], weighted transition priors [Liu+24a], or domain grounding via typed subgraphs [LDZ25]. Learned GNN-based retrievers further integrate graph topology with query representations, performing message passing over capability nodes to score candidate tools, and have been shown to improve recall and composition quality over purely vector-based baselines [NZ+25].

Our retrieval framework is inspired by HippoRAG-style [Gut+24] dense-sparse integration and Liu, Dong, and Zhang [LDZ25] graph-based exemplar generation, but it differs in both objective and design. Liu, Dong, and Zhang [LDZ25] construct and fuse a tool-dependency graph with a document knowledge graph to produce *offline* exemplar plans

for in-context planning, whereas we build a typed capability and policy knowledge graph over the CAR-bench domain and apply PPR for *online* tool recommendation and policy-aware conditioning of the agent. Concretely, our graph represents tools, parameters, outputs, policies, and examples, and our retrieval layer returns a ranked set of tools enriched with attached policies, schemas, and availability flags that are injected directly into the model's context at inference time, rather than exemplar artifacts stored for use in vector retrieval.

5 Conclusion

We adapted and extended a KG based retrieval pipeline for tool selection in LLM agents, built around PPR over a typed graph of tools, parameters, policies, and examples. Our correction steps address different weaknesses of PPR and state based agent approaches. Our results support the case for graph structured retrieval over flat capability lists in agentic systems. Future work includes testing generalization beyond the CAR-bench domain and introducing improvements to the state management of the agent.

Ethical Statement

This work does not raise significant ethical concerns. All experiments are conducted on publicly available benchmark datasets that do not contain personally identifiable or sensitive information. We do not foresee direct negative societal impacts arising from improved tool retrieval in LLM agents.

Acknowledgments

This research was conducted as part of the KIWIS research project, which focuses on advancing AI-driven knowledge management in complex production and organizational environments. The authors gratefully acknowledge the support and collaboration of all project partners and contributors involved in KIWIS.

References

[KSA26] Johannes Kirmayr, Lukas Stappen, and Elisabeth André. *CAR-bench: Evaluating the Consistency and Limit-Awareness of LLM Agents under Real-World Uncertainty*. 2026. arXiv: 2601.22027 [cs.AI]. URL: https://huggingface.co/papers/2601.22027.

[Mud+26] Sarat Mudunuri et al. "Semantic Tool Discovery for Large Language Models: A Vector-Based Approach to MCP Tool Selection". In: *arXiv preprint arXiv:2603.20313* (2026). arXiv: 2603.20313.

[NZ+25] Xiaoyi Nie, Haitao Zhang, et al. "Large Language Models Tool Retrieval and Context Compression via Dynamic Graph-Based Relation Modeling". In: *Academic Journal of Computing & Information Science* 8.7 (2025), pp. 95–104.

[Par+25] Varatheepan Paramanayakam et al. "Less Is More: Optimizing Function Calling for Llm Execution on Edge Devices". In: *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 2025, pp. 1–7.

[GS25] Tiantian Gan and Qiyao Sun. "Rag-Mcp: Mitigating Prompt Bloat in Llm Tool Selection via Retrieval-Augmented Generation". In: *arXiv preprint arXiv:2505.03275* (2025). arXiv: 2505.03275.

[Luo+25] Ziyang Luo et al. "Mcp-Universe: Benchmarking Large Language Models with Real-World Model Context Protocol Servers". In: *arXiv preprint arXiv:2508.14704* (2025). arXiv: 2508.14704.

[Liu+25] Zhiwei Liu et al. "Mcpeval: Automatic Mcp-Based Deep Evaluation for Ai Agent Models". In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. 2025, pp. 373–402.

[Mo+25] Guozhao Mo et al. "Livemcpbench: Can Agents Navigate an Ocean of Mcp Tools?" In: *arXiv preprint arXiv:2508.01780* (2025). arXiv: 2508.01780.

[Lum+25a] Elias Lumer et al. "Graph Rag-Tool Fusion". In: *arXiv preprint arXiv:2502.07223* (2025). arXiv: 2502.07223.

[LDZ25] Shengjie Liu, Li Dong, and Zhenyu Zhang. "Bridging Tool Dependencies and Domain Knowledge: A Graph-Based Framework for in-Context Planning". In: *arXiv preprint arXiv:2510.24690* (2025). arXiv: 2510.24690.

[Lum+25b] Elias Lumer et al. *Tool-to-Agent Retrieval: Bridging Tools and Agents for Scalable LLM Multi-Agent Systems*. Nov. 2025. DOI: 10.48550/arXiv.2511.01854. arXiv: 2511.01854 [cs]. (Visited on 05/11/2026).

[Liu+24a] Xukun Liu et al. "Toolnet: Connecting Large Language Models with Massive Tools via Tool Graph". In: *arXiv preprint arXiv:2403.00839* (2024). arXiv: 2403.00839.

[Liu+24b] Zhaoyang Liu et al. "Controlllm: Augment Language Models with Tools by Searching on Graphs". In: *European Conference on Computer Vision*. Springer, 2024, pp. 89–105.

[Gut+24] Bernal J Gutiérrez et al. "Hipporag: Neurobiologically Inspired Long-Term Memory for Large Language Models". In: *Advances in neural information processing systems* 37 (2024), pp. 59532–59569.